

PEO: An Ontology to Scaffold Prompt Engineering for Foundation Models

Roberto Barile¹, Simone Gramegna¹, Andrea Giovanni Nuzzolese², Donato Francioso¹ and Claudia d’Amato²

¹Dipartimento di Informatica – University of Bari Aldo Moro, Italy

²CNR – Institute of Cognitive Sciences and Technologies, Bologna, Italy

Abstract

Foundation models are revolutionizing artificial intelligence by enabling a wide variety of tasks to be solved effectively through prompting. However, designing effective prompts remains a major challenge, as the performance of foundation models is highly dependent on prompt quality. Although numerous scientific and web resources on prompt engineering are available, they are fragmented and rely on heterogeneous, non-interoperable data models, resulting in redundant and inconsistent knowledge silos. We propose PEO, an ontology that formalizes the domain of foundation models and prompt engineering to promote the interoperable representation and exchange of knowledge in this rapidly evolving field. PEO was engineered following the LOT ontology engineering methodology with the goal of supporting users in designing prompt-based solutions by identifying the most appropriate prompting techniques for a given task and foundation model. To this end, the ontology models the core concepts of the domain, including foundation models, tasks, prompting techniques, and chats documenting previous interactions with foundation models.

Keywords

Ontology, Foundation Models, Prompting

1. Introduction

AI has become an increasingly integral part of our lives, having an undeniable impact on today’s society. Currently, *Foundation Models* (FMs) [1] based on *Generative AI* are the dominant trend. They are very large deep learning models with architectures comprising billions of parameters, trained on web-scale datasets. FMs are available for different data modalities. For example, *Large Language Models* (LLMs) [2] are FMs trained on natural language data. FMs aim to be general-purpose approaches capable of solving a wide variety of tasks. Rather than designing a new model and dataset for each task, FMs enable tasks to be solved through prompting [3, 4], i.e., by crafting a model input that contains not only the input data but also the task description and auxiliary information. Any task can therefore be solved without updating the model’s parameters.

However, this makes the performance of FMs highly dependent on effective prompting. Indeed, one of the main challenges in using FMs is creating and optimizing prompts. Consequently, prompting techniques and, more generally, prompt engineering have emerged as new fields concerned with the design of effective prompts for different tasks.

With the rise of these fields, a plethora of resources has emerged on the web. They cover prompting techniques, prompts implementing such techniques, and tasks that can be effectively solved using those prompts. However, these resources are highly fragmented, and their underlying data models are not interoperable, making web-based information on these fields redundant and siloed. Consequently, relying on existing knowledge to design an effective solution is very difficult.

Therefore, we propose PEO. To the best of our knowledge, it is the first ontology that models the key concepts of prompt engineering for FMs. PEO was engineered following the LOT methodology [5] and is implemented in OWL 2. PEO is freely available online. We validated PEO by demonstrating that

WOP 2026: 17th Workshop on Ontology Design and Patterns, co-located with ISWC 2026, Bari, Italy

✉ r.barile17@phd.uniba.it (R. Barile); s.gramegna5@studenti.uniba.it (S. Gramegna); andreagiovanni.nuzzolese@cnr.it (A. G. Nuzzolese); d.francioso7@studenti.uniba.it (D. Francioso); claudia.damato@uniba.it (C. d’Amato)

🆔 0009-0007-3058-8692 (R. Barile); 0000-0003-2928-9496 (A. G. Nuzzolese); 0000-0002-3385-987X (C. d’Amato)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

it is logically sound, avoids major ontology engineering pitfalls, and fulfills the requirements defined according to its purpose of scaffolding prompt engineering.

The rest of the paper is organized as follows. Sect. 2 reviews related ontologies and other approaches to facilitating prompt engineering. Sect. 4 presents our proposal. Sect. 5 concludes the paper.

2. Related Works

Very few ontologies exist for related domains. HALO [6] represents hallucinations in LLMs. In particular, it models LLMs, hallucinations, prompts, and responses. It was created from a dataset of prompts designed to induce hallucinations. Moreover, AgentO [7] targets agentic AI systems [8]. In particular, it models agents, tasks, and tools. More general ontologies are also available. For instance, DMOP [9] targets data mining algorithms, models, and parameters. Similarly, MLOnto [10] models machine learning algorithms, frameworks, and paradigms. However, none of these ontologies specifically targets prompt engineering.

We also review prior efforts aimed at scaffolding prompt engineering. One of the first contributions in this direction is AutoPrompt [11], which generates prompts from an initial set of seed words through an optimization process that evaluates the impact of words on model behavior. Similarly, the work in [12] shows that LLMs can be effectively meta-prompted, i.e., prompted to generate a prompt. In [13], automatic prompt engineering is improved by describing the task in a more structured way. In the field of agentic AI, RePrompt [14] is a framework for automatic prompt planning as part of the decision-making process of LLM agents. Alternatively, GAAPO [15] resorts to genetic algorithms to explore the space of optimal prompts. None of these works leverages structured knowledge. Instead, OntoChatGPT [16] uses structured prompts built from ontologies. In this case, the ontology models the specific target domain. None of these works addresses the use of an ontology that models prompt engineering itself.

3. The Proposed Ontology PEO

We propose the ontology PEO, which represents knowledge about prompt engineering for FMs. To engineer PEO, we adopted LOT, a widely used ontology engineering methodology that is agile and general, i.e., it encompasses activities from other methodologies such as NeOn [17] and Methontology [18], while providing maintained tooling.

We briefly recall the phases of the methodology. It starts with *Requirements Specification*, which defines a set of functional requirements based on the domain of interest, the purpose, and the use case(s) of the ontology, as well as non-functional requirements, such as the implementation language. Next, *Ontology Implementation* is carried out for the selected set of requirements. It involves defining a conceptualization, formalizing it axiomatically, encoding it in an actual language (e.g., OWL), and validating the result with respect to the requirements. At this stage, the reuse of *Ontology Design Patterns* (ODPs) [19] and other ontologies often takes place. Subsequently, *Ontology Publication* consists of publishing the ontology and its documentation online at persistent locations. An iteration of the methodology ends with *Ontology Maintenance*, which involves bug detection and issue tracking that lead to new requirements.

In the following, we illustrate the resulting ontology by describing its requirements, high-level conceptualization (classes, object properties, and data properties), detailed formalization (including more advanced constraints) and encoding, ontology reuse strategy, validation, publication, and maintenance.

3.1. Requirements

The purpose of PEO is to formalize knowledge that facilitates prompt engineering, i.e., the design of a prompt to be submitted to an appropriate FM in order to solve a task. To this end, we envision the use case reported in Table 1.

According to this use case, we identify four main *themes* that PEO should encompass, namely:

UC	Design task solution
Description	Researchers use PEO to design a solution for a given task.
Scenario	Mike needs to solve a task by prompting a foundation model. Thus, he needs a prompting technique suitable for the task. Moreover, the task involves a specific data modality and is subject to different constraints. Another constraint concerns the model architecture, which depends on the available resources. He may also explore model families, variants, and evolutions in order to experiment with prompts. Furthermore, he relies on the analysis of prior chats to design the solution. Specifically, he can identify which models and prompting techniques are most frequently adopted for different tasks. Regarding usage patterns, he may also be interested in statistics on conversation length and in retrieving responses to specific prompts.

Table 1
Use case of PEO.

T1 Tasks: constraints and target data modalities;

T2 Foundation Models: architectures, lineage, and supported data modalities;

T3 Chats: prompts, responses, underlying models, and addressed tasks;

T4 Prompting Techniques: adoption in prompts and suitability for tasks.

In the following, we specify more precisely the notions introduced in the use case.

A task is a problem to be solved on data of a specific modality and is subject to different constraints. For instance, *text summarization* is a task on *natural language*. A task may have different constraints. More precisely, we define a constraint as a variable together with a condition consisting of an operator and a value. For instance, a constraint for *text summarization* may impose that the maximum output length is 200 characters: the variable is the output length, the value is 200 characters, and the operator is *less than or equal to*.

An FM is a deep learning model that adopts a specific neural network architecture, is able to process specific data modalities, and has been released in a particular year as part of a model family. For instance, GPT-3 is an FM based on the *decoder-only transformer*, is able to process *natural language*, and was released in 2020 as part of the GPT FM family. An architecture has a specific number of parameters. Moreover, an FM can have other FMs as variants or evolutions. For example, CODESTRAL is a variant of MISTRAL-7B adapted for source code processing, whereas GPT-3 has the evolution GPT-4, which is more recent.

A prompt is a text submitted to an FM. For instance, a prompt may be “*Classify the sentiment of the input review text into: positive, neutral, negative. Input text: <input_text>.*” A response may be “*positive.*” A chat consists of prompts and responses and targets a specific task. In particular, a chat starts with a prompt. Each prompt is followed by a response. Each response can be followed by multiple alternative prompts. The length of a chat can be measured either in terms of its duration or by the number of prompts and responses.

A prompting technique guides the definition of a prompt. For instance, *zero-shot prompting* and *chain-of-thought prompting* are prompting techniques. The *zero-shot prompting* technique is applied in the definition of the aforementioned classification prompt. Moreover, a prompting technique can be associated with the tasks for which it is considered suitable. For example, *chain-of-thought prompting* is considered suitable for tasks involving arithmetic or logical reasoning.

From these inputs, we identified the functional requirements as *Competency Questions* (CQs), grouped according to the four themes, as reported in Table 2.

ID	Competency Question
T1	<i>Tasks</i>
CQ1	What are the tasks?
CQ2	What tasks exist for a given data modality?
CQ3	What are the constraints of a given task?
T2	<i>Foundation Models</i>
CQ4	What are the possible foundation model architectures?
CQ5	What are the variants and evolutions of a given foundation model?
CQ6	What data modalities can a given foundation model process?
T3	<i>Chats</i>
CQ7	What prompts are based on a given prompting technique?
CQ8	What is the response to a given prompt?
CQ9	How many prompts and responses are there on average per chat?
CQ10	Which foundation model is a given prompt submitted to?
CQ11	How many chats were conducted per model?
CQ12	Which prompts are based on a prompting technique that is considered suitable for the targeted task?
T4	<i>Prompting Techniques</i>
CQ13	What are the prompting techniques?
CQ14	What prompting techniques are suitable for a given task?
CQ15	What prompting techniques are suitable for tasks targeting a given data modality?
CQ16	What prompting techniques are suitable for tasks with a constraint on a given variable?

Table 2
CQs of PEO.

3.2. Conceptualization

We adopted a *hybrid* (bottom-up/top-down) conceptualization approach. We first identified the core concepts emerging from the competency questions and the use case, and organized them into a high-level conceptualization. This conceptualization serves as the backbone of the ontology and captures the main entities involved in prompt engineering. Specifically, we defined four key classes: Task, FoundationModel, PromptingTechnique, and Chat.

Starting from this high-level structure, we refined each key class through a top-down process by introducing more specialized concepts and relationships. In parallel, we followed a bottom-up approach by identifying recurring concepts and relations emerging from representative examples in the domain, integrating them into the conceptual model whenever appropriate. The resulting conceptualizations were then progressively merged into the overall ontology, shown in Fig. 1.

In the following, we describe the conceptualization of each key class together with its relationships with the others.

3.2.1. Tasks

PEO models tasks together with the information required to identify the most appropriate prompting strategy and foundation model. In particular, each task is characterized by the target data modality on which it operates and by a set of constraints that define the expected properties of the solution.

To represent data modalities, PEO defines the datatype `dataModality`, whose values include natural-Language, image, video, audio, and code. Tasks are represented by the class `Task`, which is associated with the data property `targetDataModality` having range `dataModality`.

Task-specific constraints are modeled through the class `TaskConstraint`. Each constraint is described

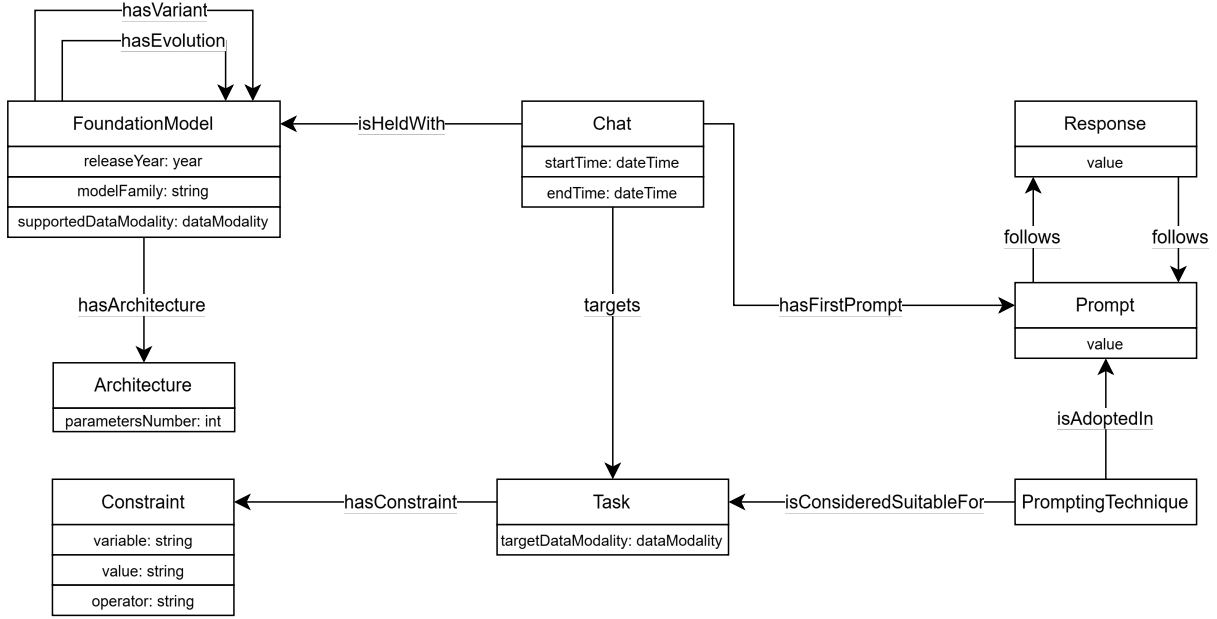


Figure 1: Conceptualization of PEO

by the data properties variable, operator, and value, all having range string. A Task is linked to one or more constraints through the object property hasConstraint.

3.2.2. Foundation Models

PEO models foundation models together with the characteristics that influence their applicability to different prompting scenarios. In particular, it represents their lineage, supported data modalities, and architectural features, enabling users to compare models and identify suitable alternatives.

A foundation model is represented by the class `FoundationModel`, which has the data property `modelFamily` with range string. The data property `supportedDataModality` associates a `FoundationModel` with a value of type `dataModality`. Moreover, PEO defines the object properties `hasVariant` and `hasEvolution` to represent relationships between foundation models. The architecture of a foundation model is represented by the class `Architecture`, which has the data property `parametersNumber` with range int. The object property `hasArchitecture` links a `FoundationModel` to its `Architecture`.

3.2.3. Chats

PEO models chats as records of interactions between users and foundation models. Besides representing prompts and responses, it captures their temporal organization, the foundation model involved, and the task addressed by the conversation.

Chats are represented by the class `Chat`, which has the data properties `startTime` and `endTime` with range `dateTime`. The classes `Prompt` and `Response` both have the data property `text` with range string. The object properties `hasPrompt` and `hasResponse` associate a chat with its prompts and responses, respectively. The object property `follows` captures the conversational flow by connecting a `Prompt` to the preceding `Response`, and vice versa. Furthermore, the object property `isHeldWithModel` links a chat to the `FoundationModel` that receives the prompts and generates the responses. Finally, the object property `targetsTask` associates a `Chat` with the `Task` it addresses.

3.2.4. Prompting Techniques

PEO models prompting techniques together with their intended usage. Specifically, it represents both the tasks for which a technique is considered suitable and the prompts in which it is adopted.

Prompting techniques are represented by the class `PromptingTechnique`. The object property `isConsideredSuitableForTask` associates a `PromptingTechnique` with a `Task`. Additionally, the object property `isAdoptedIn` links a `PromptingTechnique` to a `Prompt` defined according to that technique.

4. PEO formalization and validation

The conceptualization presented in the previous section captures the main entities and relationships of the prompt engineering domain. In this section, we describe its formalization in Description Logic (DL) and its corresponding implementation in OWL 2. The formalization complements the conceptual model by introducing logical constraints and inference rules that cannot be expressed in the graphical conceptualization alone. These axioms specify structural requirements, cardinality restrictions, property characteristics (e.g., transitivity, symmetry, etc.), and property chains that enable the inference of implicit knowledge while preserving the consistency of the ontology. Unless otherwise specified, all classes are assumed to be mutually disjoint, and the domains and ranges of properties are formalized as defined in the conceptualization. The remainder of this section presents the formalization according to the four main themes introduced previously, namely tasks, foundation models, chats, and prompting techniques. Finally, we describe the reuse approach we adopted and the ontology validation.

4.1. Formalization

In the following, we describe the translation of the conceptual model into DL axioms (and thus in OWL 2), also introducing additional constraints that were omitted in the high-level conceptualization. Unless otherwise specified, all classes in the ontology are assumed to be mutually disjoint. Furthermore, the domains and ranges are formalized as stated in the conceptualization.

4.1.1. Tasks

Tasks are constrained to operate on one or more data modalities and to be associated only with proper constraints. The existential restriction on `targetDataModality` guarantees that every task explicitly specifies at least one target data modality, while the universal restriction ensures that all associated modalities belong to the datatype `dataModality`. Similarly, the universal restriction on `hasConstraint` enforces that every constraint associated with a task is an instance of the class `Constraint`, thereby preserving the consistency of the task specification. Formally:

$$\begin{aligned}\text{Task} &\sqsubseteq (\exists \text{targetDataModality}.\text{dataModality}) \\ \text{Task} &\sqsubseteq (\forall \text{targetDataModality}.\text{dataModality}) \\ \text{Task} &\sqsubseteq (\forall \text{hasConstraint}.\text{Constraint})\end{aligned}$$

Task constraints are themselves constrained to provide a complete specification of the condition they represent. In particular, every `Constraint` must define the variable being constrained, the corresponding value, and the comparison operator used to evaluate the condition. The existential restrictions ensure that these three elements are always present, whereas the universal restrictions guarantee that their values are consistently represented as strings. Together, these axioms ensure that every constraint provides the information required to interpret and compare task requirements. Formally:

$$\begin{aligned}\text{Constraint} &\sqsubseteq (\exists \text{hasVariable}.\text{Variable}) \\ \text{Constraint} &\sqsubseteq (\exists \text{hasValue}.\text{Value}) \\ \text{Constraint} &\sqsubseteq (\exists \text{hasOperator}.\text{Operator})\end{aligned}$$

4.1.2. Foundation Models

Foundation models are constrained to be associated with well-defined architectural and functional characteristics. In particular, every `FoundationModel` must be linked to at least one architecture describing its underlying neural network and must support at least one data modality. Moreover, the ontology enforces that all architectures associated with a foundation model are instances of the class `Architecture`, and that all supported data modalities represented as individuals of the class `DataModality`. Data modalities include video, code, image, audio, natural language, etc. Similarly, the relationships describing the evolution and specialization of foundation models are constrained so that every variant and every evolution is itself a valid `FoundationModel`. These axioms guarantee that the ontology remains semantically consistent by preventing links to individuals that do not represent foundation models or architectures. Formally:

$$\begin{aligned}\text{FoundationModel} &\sqsubseteq (\exists \text{hasArchitecture}.\text{Architecture}) \\ \text{FoundationModel} &\sqsubseteq (\forall \text{hasArchitecture}.\text{Architecture}) \\ \text{FoundationModel} &\sqsubseteq (\forall \text{hasEvolution}.\text{FoundationModel}) \\ \text{FoundationModel} &\sqsubseteq (\forall \text{hasVariant}.\text{FoundationModel}) \\ \text{FoundationModel} &\sqsubseteq (\exists \text{hasSupportedDataModality}.\text{DataModality}) \\ \text{FoundationModel} &\sqsubseteq (\forall \text{hasSupportedDataModality}.\text{DataModality})\end{aligned}$$

In addition to constraining relationships, PEO also specifies cardinality restrictions for the main descriptive attributes of foundation models and their architectures. Each foundation model is required to belong to a model family, which represents a group of related foundation models belonging to the same model lineage. Additionally, a foundation model has exactly one release year and exactly an integer value to specify the number of parameters. These functional restrictions avoid ambiguous descriptions of foundation models and facilitate querying and comparison across different model families. Formally:

$$\begin{aligned}\text{FoundationModel} &\sqsubseteq (\exists \text{belongsToModelFamily}.\text{ModelFamily}) \\ \text{FoundationModel} &\sqsubseteq (= 1 \text{ releaseYear}.\text{year}) \\ \text{Architecture} &\sqsubseteq (= 1 \text{ parametersNumber}.\text{int})\end{aligned}$$

W.r.t. variants and evolutions, additional ancestor properties are introduced to capture their transitive closures. Variants are constrained as symmetric and irreflexive, whereas evolutions are asymmetric and irreflexive. These characteristics are not asserted for the transitive counterparts as it would lead to undecidability of reasoning. Also, evolutions are specific types of variants. Formally:

$$\begin{aligned}\text{Trans}(\text{hasVariantAncestor}) \\ \text{Trans}(\text{hasEvolutionAncestor}) \\ \text{hasVariant} &\sqsubseteq \text{hasVariantAncestor} \\ \text{hasEvolution} &\sqsubseteq \text{hasEvolutionAncestor} \\ \text{Sym}(\text{hasVariant}) \\ \text{Irref}(\text{hasVariant}) \\ \text{Asym}(\text{hasEvolution}) \\ \text{Irref}(\text{hasEvolution}) \\ \text{hasEvolution} &\sqsubseteq \text{hasVariant}\end{aligned}$$

4.1.3. Chats

The conversational flow is modeled as a forest-like structure alternating prompts and responses. First, in addition to the `follows` property, a `direct following` property is introduced in the formalization in order to explicitly distinguish immediate conversational transitions (which are asymmetric, irreflexive,

and transitive) from their transitive counterparts for which asymmetry can not be specified as it would lead to undecidability in reasoning. Direct follow is functional, asymmetric, and transitive. Formally:

$$\begin{aligned} \top &\sqsubseteq (\leq 1 \text{ directlyFollows}) \\ &\text{Asym}(\text{directlyFollows}) \\ &\text{Irref}(\text{directlyFollows}) \\ &\text{Trans}(\text{follows}) \\ \text{directlyFollows} &\sqsubseteq \text{follows} \end{aligned}$$

An additional property connecting chats directly to prompts and response is introduced. This turns useful to connect prompts and responses directly to FMs. This is important to fulfill the requirements, e.g., CQ10. Moreover, the property for first prompts is inverse functional, a prompt can be first prompt only of one chat. We can not make also hasMessage inverse functional as it is non-simple and this would lead to undecidability. However, this is not strictly required to guarantee that the data comply to the intended semantics as prompts and responses that follow from a first prompt are guaranteed to belong to only one chat by virtue of the inverse functionality of the first prompt. Formally:

$$\begin{aligned} \text{hasFirstPrompt} &\sqsubseteq \text{hasMessage} \\ \top &\sqsubseteq (\leq 1 \text{ hasFirstPrompt}^-) \\ \text{hasMessage} \circ \text{follows}^- &\sqsubseteq \text{hasMessage} \\ \text{hasMessage}^- \circ \text{isHeldWith} &\sqsubseteq \text{isInChatWithModel} \end{aligned}$$

Moreover, the ontology explicitly identifies the first prompt of a chat. Since every conversation starts with a prompt, the first prompt is characterized by the absence of any preceding conversational element. This constraint distinguishes the initial prompt from all subsequent prompts, which necessarily follow a response. Formally:

$$(\exists \text{hasFirstPrompt}^-. \text{Chat}) \equiv \text{Prompt} \sqcap \neg(\exists \text{directlyFollows}. \top)$$

The conversational structure of chats is further constrained to ensure a consistent alternation between prompts and responses. Every message belonging to a chat must be either a prompt or a response. Moreover, every response must directly follow exactly one prompt, while each prompt may directly follow at most one response. These axioms capture the sequential nature of prompt–response interactions while preventing inconsistent conversation structures. Formally:

$$\begin{aligned} \text{Chat} &\sqsubseteq \forall \text{hasMessage}. (\text{Prompt} \sqcup \text{Response}) \\ \text{Response} &\sqsubseteq (= 1 \text{ directlyFollows}. \text{Prompt}) \\ \text{Prompt} &\sqsubseteq (\leq 1 \text{ directlyFollows}. \text{Response}) \end{aligned}$$

Furthermore, the ontology propagates the task associated with a chat to all its constituent prompts and responses through a property chain. As a consequence, every message can be directly related to the task addressed by the conversation without requiring explicit assertions for each prompt or response. Formally:

$$\text{hasMessage}^- \circ \text{targets} \sqsubseteq \text{isInChatTargeting}$$

4.1.4. Prompting Techniques

Prompting techniques are constrained to be associated only with valid prompts and tasks. Specifically, a prompting technique can only be adopted in instances of Prompt and can only be considered suitable for instances of Task. These restrictions preserve the intended semantics of the ontology by preventing the misuse of prompting techniques in unrelated contexts. Formally:

$$\text{PromptingTechnique} \sqsubseteq (\forall \text{isAdoptedIn}. \text{Prompt})$$

$$\text{PromptingTechnique} \sqsubseteq (\forall \text{isConsideredSuitableFor.Task})$$

Finally, a second property chain infers a direct relationship between prompts and the tasks for which they adopt a suitable prompting technique. This inference makes it possible to retrieve prompts that are appropriate for a given task without explicitly asserting such relationships, thereby improving the expressiveness and queryability of the ontology. Formally:

$$\text{isAdoptedIn}^- \circ \text{isConsideredSuitableFor} \sqsubseteq \text{adoptsSuitableTechniqueFor}$$

4.2. Reuse

Ontology reuse is a key activity of the LOT methodology, as it promotes interoperability, reduces modeling effort, and encourages the adoption of established conceptualizations whenever suitable. Accordingly, we evaluated the reuse of existing ontologies related to prompt engineering and foundation models.

One candidate ontology is HALO. However, we decided not to reuse it because its primary objective is the representation of hallucinations produced by LLMs rather than prompt engineering itself. Although HALO includes concepts related to foundation models, prompts, and responses, these constructs are intentionally lightweight, as the ontology focuses on modeling hallucination phenomena. In contrast, PEO requires a richer representation of foundation models, prompting techniques, prompts, responses, and, in particular, chats, which constitute its primary domain of interest.

Another candidate is AGENTO. We also decided not to reuse this ontology because it targets the ecosystem of agentic AI systems, focusing on agents, tools, and task execution. Consequently, it does not provide the conceptual constructs required to model prompts, prompting techniques, conversational interactions, and their relationships with foundation models at the level of detail required by PEO.

Instead of reusing an entire domain ontology, PEO reuses the *Sequence Ontology Design Pattern* (ODP), which provides a generic representation of ordered sequences of objects.¹ The pattern is reused because sequencing is a fundamental aspect of the prompt engineering domain. Specifically, PEO exploits the pattern to model both the evolutionary relationships among foundation models through the object property *hasEvolution* and the ordered alternation of prompts and responses within chats. By relying on an established ODP, these sequential structures are represented according to a well-tested and reusable modeling solution rather than through ad hoc constructs.

4.3. Validation, Publication, and Maintenance

We validated the ontology by verifying that it is logically flawless, allows answering the queries, has no pitfalls, and by discussing logical and structural metrics. First, we populated the ontology with example individuals. The ontology code and all validation artefacts are available on GitHub.² Hence, we executed HERMIT that does not detect any inconsistency, unsatisfiable class or property. Next, we translated all CQs in SPARQL and successfully executed them.

We also executed OOPS! [20], a tool scanning for common pitfalls of ontology development. It signals only three minor pitfalls. First, no annotations are asserted. We will add annotations as part of the ongoing development and maintenance of the ontology. Next, the class *FoundationModel* involves itself in a subsumption axioms. We consider that it is not an error, as the axioms states that all evolution of FMs are FMs. Also inverse properties are not explicitly declared to avoid cluttering the ontology.

PEO is distributed under the *Creative Commons Attribution 4.0 International* (CC BY 4.0) license.³ To promote accessibility, reuse, and reproducibility, the ontology and its supporting resources are made publicly available through multiple complementary services.

A previous version of the ontology is available on GitHub,⁴ where its source code, issue tracking, and version history are maintained. This version is also published through the persistent identifier service

¹<https://odpa.github.io/patterns-repository/Sequence/Sequence>

²<https://github.com/rbarile17/peo>

³<https://creativecommons.org/licenses/by/4.0/>

⁴<https://github.com/simonegramegna/peo>

w3id.org,⁵ which provides a stable and dereferenceable URI for the ontology. Furthermore, it has been registered in BioPortal,⁶ facilitating its discovery and reuse within the ontology engineering community.

The ontology documentation is automatically generated from the OWL implementation using PROTÉGÉ together with the OWLDoc plug-in and is published online using VERCEL.⁷ The publication workflow is fully automated through a GitHub Action, ensuring that the online documentation remains synchronized with the ontology source.

As part of the ontology development and maintenance process, we will publish the version of PEO presented in this paper through the same infrastructure, ensuring persistent access, versioning, and reproducible documentation.

5. Conclusion

In this paper, we introduced PEO, to the best of our knowledge the first ontology specifically designed to scaffold prompt engineering for foundation models. Following the LOT ontology engineering methodology, we designed, implemented (in OWL 2), validated, and published the ontology. PEO provides an interoperable conceptualization of the key entities involved in prompt engineering, including tasks, foundation models, chats, prompts, and prompting techniques, thereby supporting the structured representation and exchange of knowledge in this rapidly evolving domain.

As future work, we plan to populate PEO with publicly available information about foundation models, prompting techniques, tasks, and representative chats to build a knowledge graph supporting prompt engineering. Furthermore, we intend to investigate the use of AI agents [8] that exploit the ontology to assist users in selecting foundation models, prompting techniques, and prompts that are most suitable for a given task.

References

- [1] S. Yin, C. Fu, S. Zhao, K. Li, X. Sun, T. Xu, E. Chen, A survey on multimodal large language models, *National Science Review* 11 (2024) nwae403.
- [2] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang, et al., A survey on evaluation of large language models, *ACM transactions on intelligent systems and technology* 15 (2024) 1–45.
- [3] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, G. Neubig, Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing, *ACM computing surveys* 55 (2023) 1–35.
- [4] G. Marvin, N. Hellen, D. Jjingo, J. Nakatumba-Nabende, Prompt engineering in large language models, in: *International conference on data intelligence and cognitive informatics*, Springer, 2023, pp. 387–402.
- [5] M. Poveda-Villalón, A. Fernández-Izquierdo, M. Fernández-López, R. García-Castro, LOT: An industrial oriented ontology engineering framework, *Engineering Applications of Artificial Intelligence* 111 (2022) 104755.
- [6] N. Nananukul, M. Kejriwal, Halo: an ontology for representing and categorizing hallucinations in large language models, in: *Disruptive technologies in information sciences VIII*, volume 13058, SPIE, 2024, pp. 86–100.
- [7] A. Ekelhart, K. Kurniawan, F. J. Ekaputra, E. Kiesling, AgentO: An Ontology for Modeling Agentic AI Systems, in: *European Semantic Web Conference*, Springer, 2026, pp. 298–320.
- [8] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, React: Synergizing reasoning and acting in language models, *arXiv preprint arXiv:2210.03629* (2022).

⁵<https://w3id.org/peo>

⁶https://bioportal.bioontology.org/ontologies/PEO_ONTOLOGY

⁷<https://peoontology.vercel.app/>

- [9] C. M. Keet, A. Ławrynowicz, C. d'Amato, A. Kalousis, P. Nguyen, R. Palma, R. Stevens, M. Hilario, The data mining optimization ontology, *Journal of web semantics* 32 (2015) 43–53.
- [10] J. Braga, J. L. Dias, F. Regateiro, A machine learning ontology, *Frenxiv Pap* (2020).
- [11] T. Shin, Y. Razeghi, R. L. L. Iv, E. Wallace, S. Singh, AutoPrompt: Eliciting knowledge from language models with automatically generated prompts, in: *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, 2020, pp. 4222–4235.
- [12] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, J. Ba, Large language models are human-level prompt engineers, in: *The eleventh international conference on learning representations*, 2022.
- [13] Q. Ye, M. Ahmed, R. Pryzant, F. Khani, Prompt engineering a prompt engineer, in: *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 355–385.
- [14] W. Chen, S. Koenig, B. Dilkina, RePrompt: Planning by automatic prompt engineering for large language models agents, *arXiv preprint arXiv:2406.11132* (2024).
- [15] X. Sécheresse, J.-Y. Guilbert-Ly, A. Villedieu de Torcy, GAAPO: genetic algorithmic applied to prompt optimization, *Frontiers in Artificial Intelligence* 8 (2025) 1613007.
- [16] O. Palagin, V. Kaverinskiy, A. Litvin, K. Malakhov, OntoChatGPT information system: Ontology-driven structured prompts for ChatGPT meta-learning, *arXiv preprint arXiv:2307.05082* (2023).
- [17] M. C. Suárez-Figueroa, A. Gómez-Pérez, M. Fernandez-Lopez, The NeOn Methodology framework: A scenario-based methodology for ontology development, *Applied ontology* 10 (2015) 107–145.
- [18] M. Fernandez-Lopez, A. Gomez-Perez, N. Juristo, Methontology: from ontological art towards ontological engineering, in: *Proceedings of the AAAI97 Spring Symposium*, Stanford, USA, 1997, pp. 33–40.
- [19] A. Gangemi, V. Presutti, Ontology design patterns, in: *Handbook on ontologies*, Springer, 2009, pp. 221–243.
- [20] M. Poveda-Villalón, A. Gómez-Pérez, M. C. Suárez-Figueroa, OOPS! (OntOlogy Pitfall Scanner!): An On-line Tool for Ontology Evaluation, *International Journal on Semantic Web and Information Systems (IJSWIS)* 10 (2014) 7–34.